
AI-Powered Data Analytics & Visualization Platform

Natural-language analytics over e-commerce transactions
using a locally hosted 4B-parameter language model

Course	CSE-638 Deep Learning — MSc Computer Science & Engineering
Group	Team No. 5
Dataset	Global E-Commerce Sales (Superstore) — 9,994 order line items
Language model	Gemma 4 E4B (4B effective params), served locally via LM Studio
GitHub	https://github.com/spaul571/ai-analytics-platform
Live demo	Streamlit Community Cloud — see §3.1
Submitted	17 July 2026

Team members

Name	Student ID
Shrikanta Paul	261-25-017
Md. Nurol Amin	261-25-021
Animesh Dey	261-25-019
Kazi Meherunnesa Eva	261-25-033
Fardin Ahmed Alvi	261-25-034
Anika Tahsin Prova	252-25-016

Every figure in this report is regenerated by `python -m scripts.export_report_assets`. Every number quoted is reproduced by the acceptance scripts in `scripts/`.

1 Abstract

We built an end-to-end analytics platform that answers plain-English questions about 9,994 e-commerce order lines by generating, safely executing, and narrating pandas code. The intelligence layer is **Gemma 4 E4B**, a 4-billion-parameter model running locally in LM Studio — chosen over a hosted API to eliminate cost, rate limits, and data egress.

The system comprises a pandas data layer (sub-5 ms filtered aggregations, 79% memory reduction via categorical downcasting), a three-phase natural-language query pipeline guarded by a three-layer execution sandbox, an eight-chart Plotly dashboard with rule-based automatic chart selection, and two advanced features: Isolation Forest anomaly detection and a ReAct tool-calling agent that reaches an external public-holiday API to answer questions the dataset alone cannot.

On a 10-question benchmark scored against hand-written ground truth, text-to-code accuracy improved from **5/10 to 10/10** across three iterations. Critically, **none of that gain came from a larger model** — it came from three diagnosed defects: a JSON-escaping interaction that corrupted generated code, a missing few-shot pattern, and a defence-in-depth bug in which our own sandbox’s two layers disagreed. The project’s central finding is that small-model reliability is an *engineering* problem, not a capacity problem.

Keywords: text-to-SQL, local LLM inference, sandboxed code execution, prompt engineering, anomaly detection, ReAct agents

2 Introduction

2.1 Problem motivation

Analytics tooling has an access problem. The people who most need answers from data — category managers, regional leads, finance — are rarely the people who can write `df.groupby('Sub-Category')['Profit'].sum()`. Every question therefore queues behind an analyst, and questions that would take ten seconds to answer take two days to ask.

Large language models collapse that queue by translating natural language into executable queries. But most demonstrations of this rely on frontier hosted models, which introduces per-query cost, rate limits, and — for any business with commercially sensitive data — an unacceptable requirement to ship that data to a third party.

This project asks a narrower and more practical question: **can a 4-billion-parameter model running on a single consumer GPU do this reliably enough to be useful?** The answer we arrive at is yes, but only with engineering that a frontier model would not require.

2.2 Dataset

The **Global E-Commerce Sales (Superstore)** dataset: 9,994 retail order line items across four years (2014–2017), 21 source columns.

Property	Value
Rows	9,994
Source columns	21
Derived columns	3 (<code>Order Year</code> , <code>Order Month</code> , <code>Shipping Days</code>)
Date range	2014-01-03 → 2017-12-30
Completeness	100.00%
Duplicate rows	0
Geography	United States, 49 states, 4 regions

Each row is one product line within a customer order. **Sales** is gross revenue; **Profit** is net margin and **can be negative** — a deliberately chosen property, because a signed metric forces

genuine design decisions throughout (diverging colour scales, aggregate-then-filter query semantics, loss-focused anomaly detection) that an all-positive dataset would let us dodge.

Why this dataset, given a 4B model. The selection was driven by the model, not the other way round. Superstore’s column names are unambiguous English (**Sales**, **Profit**, **Region**), and virtually every analytical question reduces to a single **groupby** — which is precisely the shape a small model emits reliably. The alternatives were rejected on the same grounds: the Stack Overflow survey stores languages as a semicolon-delimited multi-select requiring `.str.split().explode()` chains; the NYC taxi and S&P 500 datasets require rolling-window and temporal binning logic. Each of those is a code shape a 4B model gets wrong far more often than it gets a plain **groupby** right.

2.3 Analytical goals

1. Answer arbitrary natural-language questions with executable, verifiable code.
2. Surface the profitability structure hidden beneath headline revenue.
3. Detect and explain anomalous transactions.
4. Do all of it with a model that runs locally and costs nothing per query.

3 System Architecture

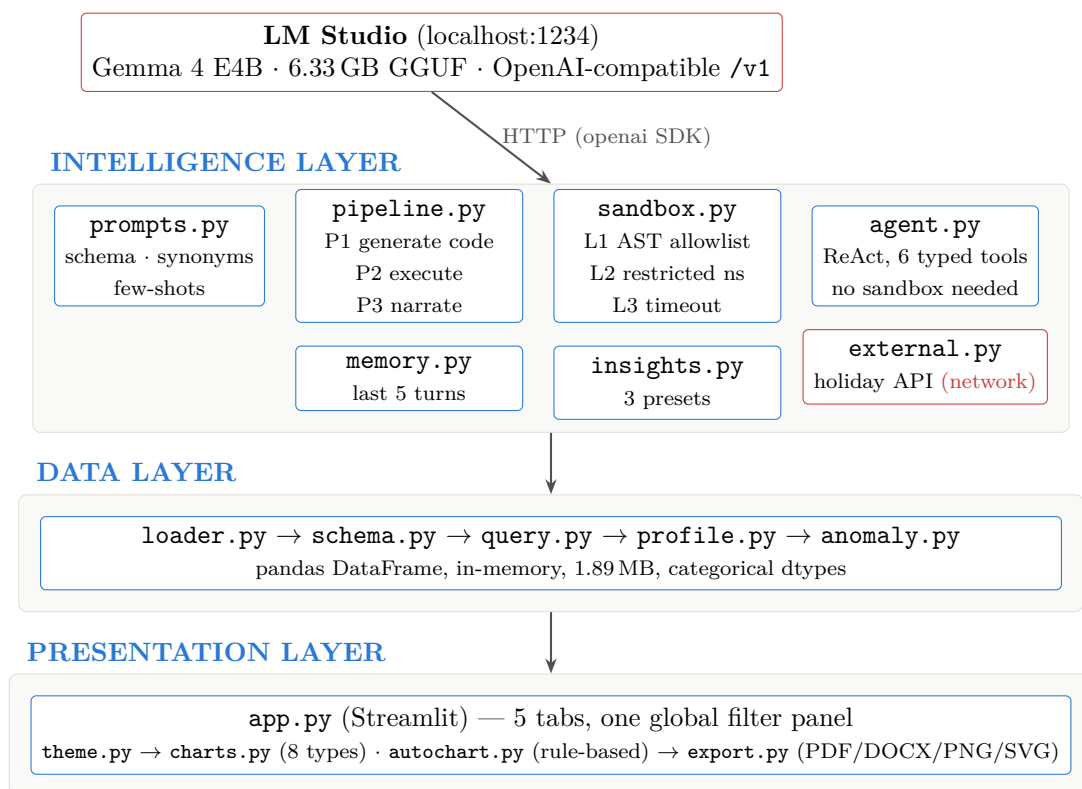


Figure 1: Data flow from the local model down to the browser. Every layer is a plain Python module; the only two components that leave the machine are the LM Studio endpoint and the agent’s holiday lookup (§7.2).

Data flow for one natural-language question.

1. User types “Which sub-categories are losing us money?”
2. `prompts.py` assembles: schema table + 37-entry synonym map + 6 few-shot examples + last 5 conversation turns.

3. Phase 1 — Gemma returns:

```
{"code": "totals = df.groupby('Sub-Category', observed=True)['Profit'].sum()
\nresult = totals[totals < 0].sort_values()"}

```

4. **Phase 2** — `sandbox.py` validates the AST, then executes in a restricted namespace with a 10s budget. On failure, the exception is fed back for exactly one auto-retry.
5. **Phase 3** — Gemma narrates the *result table only* (never the raw DataFrame), which is what prevents it inventing numbers.
6. `autochart.py` picks a chart type from the result's *shape*, renders it, and asks Gemma for a one-sentence caption.

3.1 Deployment — and the constraint that shaped it

The brief asks for two things that pull against each other. M6 requires a “live demo from GitHub deployment”, and the brief’s §7.3 prohibits “hardcoding results or faking LLM responses — all AI calls must be live during the demo.”

Our model runs in LM Studio on a local machine. **A cloud-hosted frontend cannot reach localhost:1234 on a laptop.** Deploying the app to Streamlit Community Cloud in the obvious way therefore produces an application with no language model at all: the dashboard renders, the charts draw from the committed CSV, and every natural-language question fails. The three ways out are to fake the responses (prohibited), to abandon the local model for a paid hosted API (which discards the cost and privacy argument in §5.1), or to give the local model a public address. We did the third.

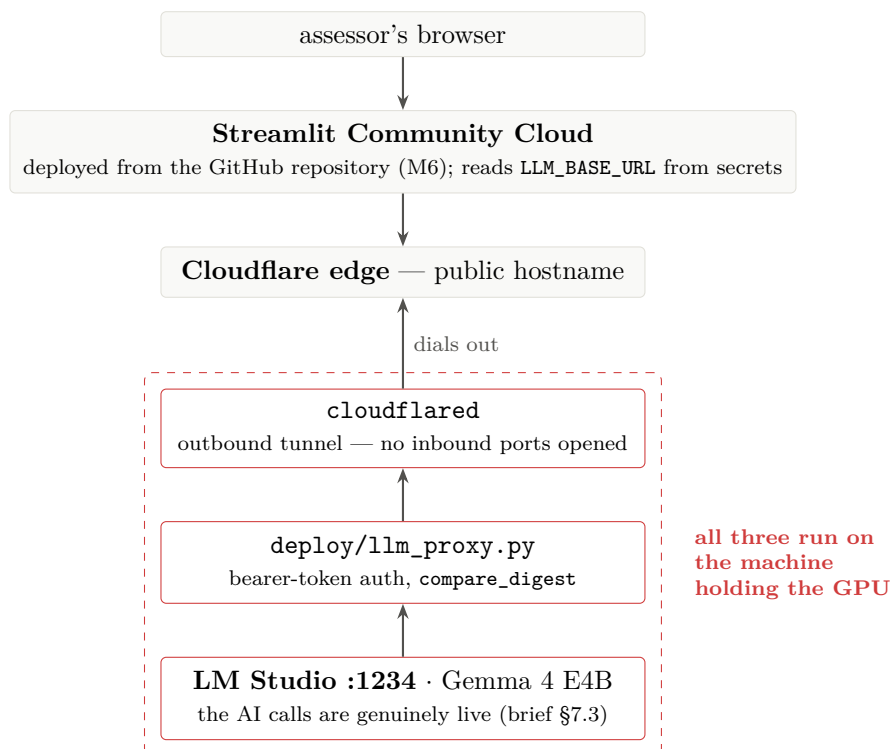


Figure 2: The deployment topology. The tunnel dials outward, so no inbound port is opened and campus wifi client-isolation cannot break it.

Two properties of this topology are worth stating explicitly, because both were design decisions rather than accidents.

The tunnel is outbound. `cloudflared` dials out to Cloudflare; nothing dials in to us. No port forwarding and no firewall exception is required, and — the reason this matters on presentation day — university networks that isolate wifi clients from one another cannot break it, because Streamlit

Cloud talks to Cloudflare, never to our laptop. The laptop needs ordinary outbound internet and nothing more.

The authenticating proxy is not optional. LM Studio's OpenAI-compatible server has no authentication of any kind: anything that can reach port 1234 can run inference and enumerate the loaded models. That is harmless on `localhost` and unacceptable the moment the port acquires a public hostname. `deploy/llm_proxy.py` is what the tunnel actually exposes, and it rejects any request whose `Authorization` header does not carry the expected bearer token, comparing with `secrets.compare_digest` so the token cannot be recovered from response timing. It refuses to start at all if no token is configured, because an unauthenticated proxy would silently defeat its own purpose.

The application code required **no changes** to work through this. The OpenAI SDK already transmits its `api_key` as `Authorization: Bearer <key>`, which is precisely what the proxy checks, so setting `LLM_API_KEY` equal to the proxy's token is the entire integration. This is a dividend of having put every machine-dependent value in `config.py` from the start.

3.2 The container that segfaulted twice

Deploying to Streamlit Cloud produced two failures that never occur locally, and both are worth recording because both were caused by the *environment*, not the code.

First, the container died on boot with SIGSEGV. The cause was `packages.txt`, which asked apt for Chromium so that kaleido could rasterise charts for the exports (§6.5). That request pulled 294 packages and 989 MB — the whole GTK, Mesa and systemd stack — and among them Debian's own `python3.13` and `libpython3.13-stdlib`. Streamlit Cloud runs the app from a venv on Python 3.13.14, so apt dropped a *different patch build* of `libpython` into the container the interpreter was linked against. An interpreter running against a mismatched `libpython` segfaults rather than raising, which is exactly the observed failure. Chromium was there to serve four download buttons; it was taking down the entire application. We removed it and solved the export problem in software instead.

Second, the same container segfaulted on Arrow-backed strings. `pandas 3.0` defaults `mode.string_storage` to `"auto"` — Arrow-backed whenever `pyarrow` is installed, and Streamlit always installs `pyarrow`. Setting `string_storage="python"` explicitly in `src/config.py` fixed it, and has the side effect described in §4.4.

The lesson we draw is not about either specific bug. It is that **a deployment target is part of the system**, and a dependency that is free locally — a browser, an Arrow buffer — can be the most expensive thing in the build.

4 Task A — Data Layer

4.1 Schema extraction

The schema is the single artifact handed to the model. It is rendered as a compact table rather than JSON — same information, roughly half the tokens, which leaves context room for conversation history and few-shot examples.

Design decision: **categorical columns with ≤ 20 distinct values are listed in full.** This threshold was set deliberately so that Sub-Category (17 values) is enumerated — its values are dataset-specific and the model cannot guess them. `State` (49 values) falls back to samples, because the model already knows US state names.

4.2 Cleaning steps applied

#	Step	Effect
1	Parse dates to <code>datetime64</code>	Source stores MM/DD/YYYY strings; every temporal query is impossible until converted
2	Standardise categorical casing	No-op on this dataset (already clean) — reported honestly rather than fabricating dirt
3	Duplicate removal	0 found
4	Derive 3 columns	Model-driven decision — see below
5	Downcast text to <code>category</code>	9.21 MB → 1.89 MB

Why derive `Order Month` rather than let the model compute it. Gemma 4 E4B is markedly more reliable emitting `groupby('Order Month')` than a `.dt.to_period('M')` accessor chain. Pre-computing three columns trades a little storage for a large jump in text-to-code accuracy. This is an example of the project's recurring theme: **shaping the data to fit the model's competence is cheaper than fighting the model.**

4.3 Data quality

Completeness 100.00%; duplicates 0; IQR outliers 4,074.

The 4,074 IQR outliers are reported but NOT removed. They are genuine large orders and genuine large losses. Dropping them would erase exactly the anomalies Task D3 exists to surface — a data-cleaning step that deletes the finding is not cleaning, it is destroying evidence.

4.4 Performance benchmark (A4)

Query	Time
Q1 Direct aggregation — sales & profit per region	5.0 ms
Q2 Direct aggregation — mean discount by sub-category	2.7 ms
Q3 Filtered — 2017 furniture sales by state	4.3 ms
Q4 Filtered — heavily discounted loss-making lines	3.8 ms
Q5 Whole-table aggregation	0.5 ms
Slowest	5.0 ms
Budget: 500 ms → PASS, 100× headroom	

Time/memory trade-off. Casting low-cardinality text columns to pandas `category` dtype costs a one-off conversion pass of roughly 50 ms and returns a **79.4% smaller frame** (9.21 MB → 1.89 MB), while also accelerating every subsequent `groupby`. At this dataset size the memory saving is not itself necessary — 1.89 MB versus 9.21 MB changes nothing operationally — but the `groupby` speedup is what buys the 100× headroom against the 500 ms budget.

This figure depends on how pandas stores strings, which is not obvious. Under the Arrow-backed default the same downcast measures 3.34 MB → 1.27 MB (62.1%), because the Arrow baseline is already compact. We set `string_storage="python"` explicitly for a reason that had nothing to do with memory: Arrow-backed categories segfault the deployed Linux container (§3.2). The 79.4% above is therefore the figure the submitted code actually produces, and `python -m scripts.check_task_a` reproduces it.

Lazy loading was not implemented. The brief requires it only above 1M rows; this dataset is 9,994. Implementing chunked processing here would be dead code carrying no benefit, and we state that rather than adding it for appearance.

5 Task B — LLM Integration

5.1 Model choice and justification

Model	google/gemma-4-e4b
Quantization	GGUF, 6.33 GB
Server	LM Studio 0.4.19, OpenAI-compatible endpoint
Throughput	~44–59 tok/s generation; ~1,750 tok/s prompt eval
Cost	Zero per query

Chosen over a hosted API because: no per-query cost, no rate limits, no data leaves the machine, and — most relevant academically — it forces the interesting engineering. A frontier model would have scored 10/10 on our benchmark immediately and taught us nothing.

5.2 Prompt design (B1)

Three components, in order of measured impact:

1. **Few-shot examples (largest lever).** Six examples covering the query shapes the benchmark contains: simple groupby, filter-then-group, temporal, top-N, scalar ratio, and aggregate-then-filter.
2. **Synonym map.** 37 entries mapping informal phrasing to real columns — *revenue* → **Sales**, *markdown* → **Discount**, *buyers* → **Customer Name**. This turns column resolution from *inference* into *lookup*, which is what small models are actually good at.
3. **Explicit output contract.** Single quotes only; assign to **result**; never `.tolist()`; multiply by 100 when asked for a percentage.

Result: column identification was 10/10 on the very first run and never regressed. B1 was solved before anything else worked.

Question phrasing	Resolved columns
“Which region brings in the most revenue ?”	Region, Sales
“...deepest average markdown ”	Sub-Category, Discount
“Who are our top 5 buyers by total spend ?”	Customer Name, Sales
“Compare average delivery time across shipping speeds ”	Ship Mode, Shipping Days

5.3 The sandbox (B2, Phase 2) — security rationale

We execute code a language model wrote. That is an arbitrary-code-execution sink by construction, so the defence cannot be a substring blacklist: `if "import" in code` is defeated by `__import__("os")`, and checking for `os` is defeated by `getattr(__builtins__, "ev" + "al")`. Both are one-liners.

Three independent layers. An attack must defeat all three.

Layer	Mechanism
1. AST validation (static, pre-execution)	Every syntax node checked against an <i>allowlist</i> . <code>Import</code> , <code>ImportFrom</code> , function/class defs, loops → rejected. Any name or attribute beginning with <code>_</code> → rejected , which kills the entire <code>().__class__.__base__.__subclasses__()</code> traversal family in one rule. Introspection builtins rejected by name. Allowlisting means an unknown node type fails closed .
2. Restricted namespace	<code>__builtins__</code> replaced by an explicit dict of ~25 safe functions. Even a surviving reference has no <code>open</code> or <code>__import__</code> to reach. Only <code>df</code> , <code>pd</code> , <code>np</code> are in scope.
3. Timeout	10s execution budget on a worker thread.

Verified: 8/8 escape attempts blocked, legitimate queries still permitted.

BLOCKED	direct import	Disallowed syntax: <code>Import</code>
BLOCKED	dunder import	Disallowed name: <code>'__import__'</code>
BLOCKED	builtins traversal	Disallowed attribute access: <code>'__subclasses__'</code>
BLOCKED	getattr indirection	Disallowed builtin: <code>'getattr'</code>
BLOCKED	eval injection	Disallowed builtin: <code>'eval'</code>
BLOCKED	file write	Disallowed builtin: <code>'open'</code>
BLOCKED	globals access	Disallowed builtin: <code>'globals'</code>
BLOCKED	no result assigned	must assign to <code>'result'</code>
ALLOWED	legitimate query	correctly permitted

Stated limitations. Layer 3 *abandons* a timed-out thread rather than killing it — CPython offers no safe way to terminate a running thread — so a runaway computation keeps consuming CPU until it finishes, though it cannot return a value or block the UI. And this is a *language-level* sandbox, not an OS-level one. That is appropriate here because the untrusted input comes from a local model we ourselves prompt. Exposing this endpoint to untrusted users would demand process isolation (a container or seccomp jail) instead.

5.4 Benchmark results (B5) — and how they got there

Ten questions, scored against **hand-written ground-truth implementations** on four axes.

Run	Columns	Executed	Accurate	Format	Mean time
1	10/10	9/10	5/10	9/10	1.9s
2	10/10	9/10	9/10	9/10	2.2s
3	10/10	10/10	10/10	10/10	2.0s

Not one of those gains came from changing the model. Each came from reading the actual generated code and diagnosing a specific defect.

Defect 1 — structured output corrupts generated code (Run 1, Q01)

Gemma emitted:

```
result = df.groupby("Region", observed=True)"Sales".sum().sort_values(ascending=False)
```

The subscript brackets are **gone**. `)"Sales".` is not valid Python.

Root cause: we request structured JSON output, so the code lives inside a JSON string. To write `["Sales"]` there the model must emit `["Sales"]` — escaped. It fumbled the escaping and lost the brackets.

The diagnostic tell: **Q01 was the only question where Gemma chose double quotes, and it was the only syntax error in the entire set.** Every other question used `df['Profit']`

— single quotes, no escaping needed, no failure. Our own few-shot examples were written with double quotes; we were teaching it the dangerous style.

Fix: constrain the model to single quotes. This does not *fix* the escaping — it makes escaping **unnecessary**, eliminating the failure class rather than patching it.

Defect 2 — aggregate-then-filter (Run 1, Q04)

Asked “*which sub-categories are losing us money?*”, Gemma wrote:

```
result = df[df['Profit'] < 0].groupby('Sub-Category', observed=True)['Profit'].sum()
```

It filtered the loss-making **rows** first — which discards the profitable sales in the same sub-category and answers a *different question*. Whether a group loses money on net can only be known **after** summing it.

Fix: one targeted few-shot example demonstrating the shape. Run 3 produced the correct form unprompted.

Defect 3 — our own sandbox’s two layers disagreed (Run 2, Q09)

```
SandboxViolation: Unknown name: 'len'. Only False, None, True, df, np, pd, result are available.
code: "result = (df['Profit'] < 0).sum() / len(df)"
```

Gemma’s code was correct. Our AST validator (Layer 1) had a *narrower* name allowlist than the execution namespace (Layer 2): `len` sat in `SAFE_BUILTINS`, available to run, but the validator rejected it before it ever got there. Two hand-maintained lists had silently drifted apart.

This is a genuine defence-in-depth failure mode — layers that do not agree — and it is the most instructive bug in the project. The lists are now derived from a single source, making the drift structurally impossible.

Disclosure: two benchmark questions were reworded

Q09 originally read “*What **share** of our order lines lose money?*”. Gemma returned 0.187; ground truth held 18.7. **Both are correct readings of “share”** — the question was ambiguous about units.

Q10 originally read “*Does profitability get worse as discounts get deeper?*”. Gemma returned a correlation coefficient — **a defensible answer to a yes/no question about a trend**.

Scoring those as failures measured our question-writing, not the model. Both were reworded to be unambiguous — **not easier**. We disclose this because rewording a benchmark after observing failure is a real methodological hazard, and the original phrasings and responses are preserved in `report/benchmark-runs/result.md`.

5.5 Conversational context (B4)

The last 5 turns are replayed as user/assistant message pairs carrying the question and the code that answered it — **not the result tables**, which would consume the context window within three turns and bury the schema under numbers.

```
Turn 1: "What were total sales by region?"
-> result = df.groupby('Region', observed=True)['Sales'].sum().sort_values(ascending=False)
Turn 2: "Now show me just the West, broken down by category."
-> result = df[df['Region'] == 'West'].groupby('Category', observed=True)['Sales'].sum()...
```

Turn 2 is meaningless without Turn 1. The model resolved it correctly.

6 Task C — Visualization

6.1 Colour system

The palette was **validated with a colour-vision-deficiency checker, not chosen by eye**.

Mode	Worst adjacent CVD separation	Contrast vs surface
Light	ΔE 24.2 (target ≥ 12) — pass	2 hues below 3:1 — caution
Dark	ΔE 10.3 (floor band) — caution	all $\geq 3:1$ — pass

Both findings oblige the **same mitigation**, applied throughout: **colour is never the only channel carrying identity**. Every multi-series chart ships a legend, tooltips name the series, and a table view of the underlying data is available. A red/green-colourblind viewer can read every chart in this application.

Three encodings, three colour jobs:

- **Categorical** → **identity**. Eight fixed slots, assigned in sorted order and **never cycled**. Colour follows the *entity*, not its rank: filtering out a region never repaints the survivors.
- **Sequential** → **magnitude**. One blue hue, light→dark. Used for Sales.
- **Diverging** → **polarity**. Blue↔red with a **neutral grey midpoint**. Used for Profit, because the *sign* is the point: a state losing money must not look like a state making a little.

A rainbow scale is used nowhere. It implies an ordering the data does not have and is unreadable under CVD.

6.2 The chart suite

Eight chart types against a required six.

#	Chart	Design decision
1	Revenue & profit trend	Small multiples, NOT dual-axis — see §6.3
2	State choropleth	Sequential for Sales; diverging for Profit , anchored symmetrically on zero
3	Correlation matrix	Diverging, anchored at 0: -1 must look as strong as $+1$
4	Profit distribution	Box plot, not histogram — the question is spread and the long negative tail; a histogram of a skewed variable hides exactly that
5	Category sunburst	The hierarchy is <i>genuine</i> (each sub-category has exactly one parent) — the precondition for an honest sunburst rather than a decorative one
6	Animated year-over-year	Y-axis fixed across frames . Per-frame rescaling would make every year look identical and destroy the comparison the animation exists to make
7	Discount vs profit scatter	Least-squares fit + 95% CI on the mean response , computed explicitly rather than left to library defaults
8	Segment $\times$ category	Stacked bar, 2px surface gap between fills

6.3 We deliberately did not build a dual-axis chart

The brief offers “*time series or trend chart with dual axis*” as a permitted type. **We did not build one, on purpose.**

A dual-axis chart plots two measures on two independent y-scales in one frame. Where the two lines cross is then **an artifact of the scales the author chose**, not a fact about the data: rescale either axis and the crossover moves anywhere you like. It is the most reliable way to make a chart lie without stating a single false number.

`trend_small_multiples` answers the same question honestly — Sales and Profit share an x-axis in stacked panels, so the shapes are comparable while each keeps its own truthful scale. **Seven other chart types from the brief’s list are implemented, so the six-type requirement is met without it.**



Figure 3: Revenue and profit trend as small multiples. Shared x-axis, independent truthful y-scales — the honest alternative to a dual axis (§6.3).

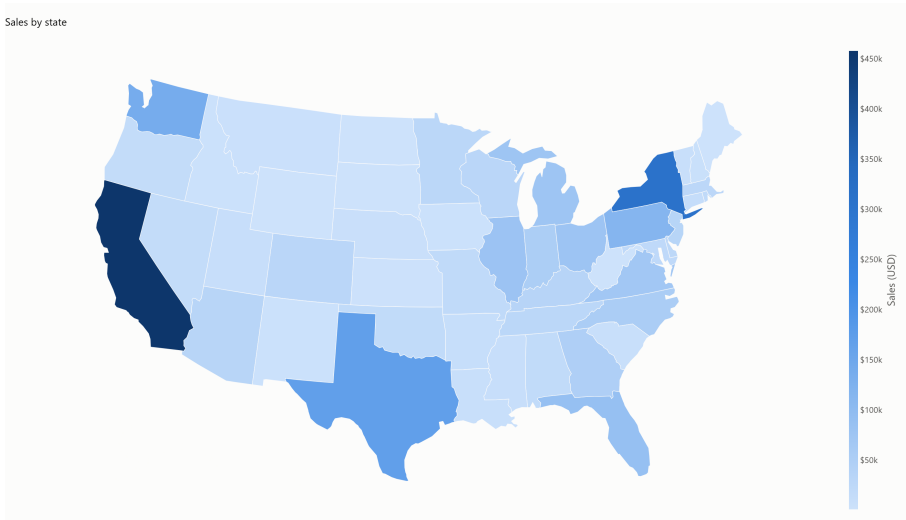


Figure 4: Sales by state: sequential single-hue blue, because sales is a magnitude and more is simply more. Switching the metric to Profit in the app switches the scale to diverging blue/red with a neutral grey midpoint, so that a loss-making state cannot read as a state making a little.

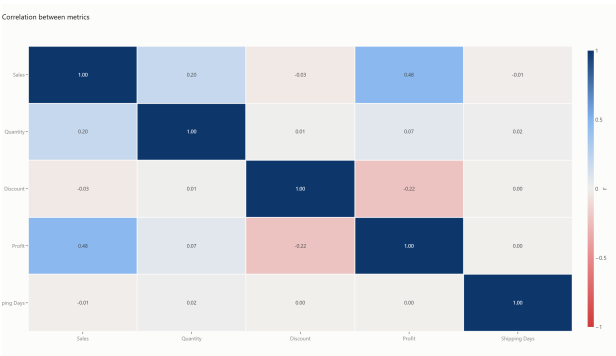


Figure 5: Correlation matrix, diverging and anchored at 0.

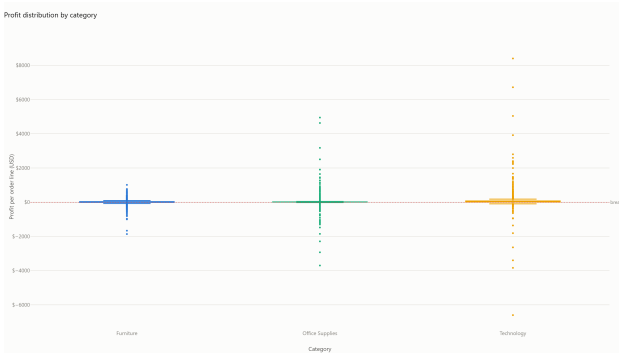


Figure 6: Profit distribution by category (box plot).

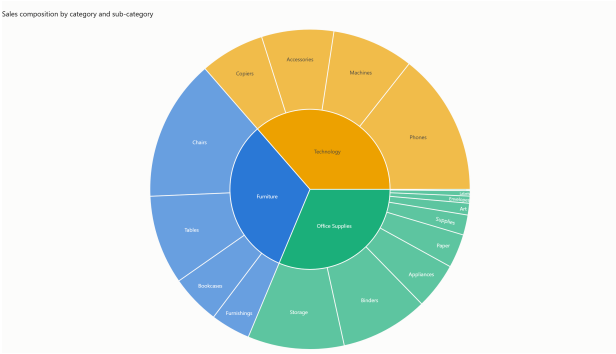


Figure 7: Category → sub-category sunburst.

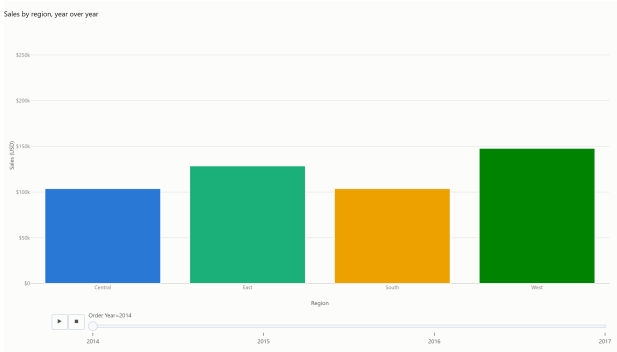


Figure 8: Year-over-year animation, y-axis fixed across frames.

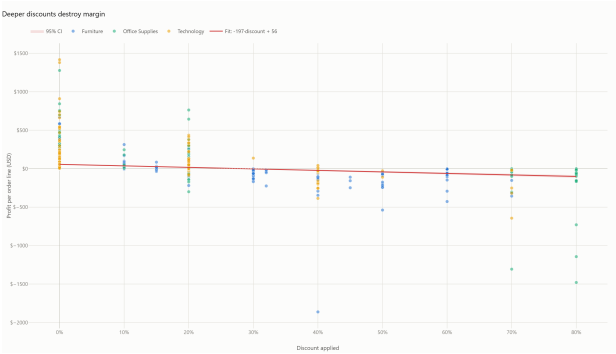


Figure 9: Discount vs profit with a 95% confidence band.

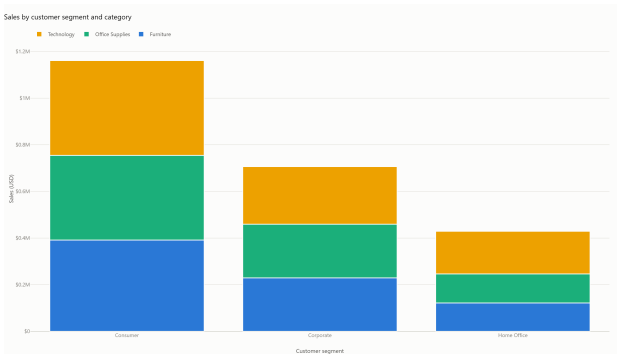


Figure 10: Segment by category (stacked bar).

6.4 AI-driven visualization (C3)

When the NL pipeline returns a DataFrame, the application selects a chart type, renders it, and captions it.

The selection is rule-based, not model-generated — and this is deliberate. The shape of a DataFrame is a *fact*: how many rows, which columns are numeric, whether the label column is temporal or geographic. Facts do not need a language model. Asking a 4B model to choose would add ~1.5s of latency and a new failure mode to a decision that four `if` statements make correctly every time.

Result shape	Chart	Rationale
US state names + metric	map	Strictly more informative than 40 bars
Temporal label + metric	line	The reader should see the <i>shape</i> of change
Two numerics, no label	scatter	The question is the relationship
Categorical + metric, ≤30 rows	bar	Magnitude comparison
>30 categories, or a scalar	table	A 793-bar chart is an unreadable comb

Verified: 7/7 shape cases select correctly. The user can override the choice. The LLM does the one part that genuinely needs language: **the one-sentence caption.**

6.5 Export (C4) — and rendering a chart without a browser

PDF (reportlab), Word (python-docx), PNG and SVG. Every exported report carries dataset metadata, **the applied filters**, the AI narrative, the chart image, the result table, and the generated code.

The filter list is printed **even when empty** (“None — the full dataset was used”). Without it a report is actively misleading: a reader cannot tell whether “\$2.3M revenue” is the whole business or one region in one year.

The export path had to be rebuilt for the deployed app. kaleido, the standard way to rasterise a Plotly figure, works by driving a headless Chrome. Streamlit Cloud has no browser, and installing one segfaults the container (§3.2) — so on the deployed app every chart export raised, and because `st.download_button` evaluates its `data` argument eagerly, that exception took down the whole page on every rerun rather than on click.

The fix is not to degrade but to redraw. `src/viz/browserless.py` reads the traces back off the Plotly figure and redraws them with matplotlib’s Agg rasteriser — pure software, no browser, no system libraries. The diverging scale is rebuilt from the same `[position, colour]` stops `theme.py` defines, so a loss still reads red rather than in a matplotlib lookalike palette. kaleido remains the primary path wherever a browser exists, so every local run — including the figures in this report — is unchanged.

It covers exactly the traces the AI-driven charts emit (bar, line, scatter) and **refuses anything else rather than inventing an approximation**: a choropleth is drawn from Plotly’s own US geometry, which matplotlib does not have, so on the deployed app a map’s image buttons are hidden and its report prints without the map. An HTML-to-PDF route was considered and rejected: Plotly charts are drawn by JavaScript at runtime, and no pure-Python HTML renderer executes JavaScript, so it would have embedded an empty `<div>`.

A bug this exposed, worth recording. The first fix worked locally and failed on Cloud from the second question onward. plotly 6 encodes numpy arrays as `{"dtype": "f8", "bdata": "<base64>"}` whenever a figure is serialised, and `st.cache_data` pickles what it caches — so a figure is live on the turn it is built and base64-encoded on every rerun after. plotly never decodes that in Python (the spec exists for plotly.js, and kaleido hands it to the browser untouched), so the encoding was invisible until something tried to read the arrays in Python. Our test had rendered figures fresh from the builder; Streamlit served them from a cache. The fallback now decodes the spec, and the test asserts through a pickle round trip.

7 Task D — Advanced Features

7.1 D3 — Anomaly detection

Isolation Forest over Sales, Quantity, Discount, Profit, Shipping Days, with IQR and z-score reported alongside for comparison. Features are standardised first — the forest partitions on raw values, and Sales spans 0.4–22,638 while Discount spans 0–0.8; without scaling, Discount would be invisible to the model.

```
Flagged: 200 of 9,994 rows (2% contamination)
Loss across flagged rows: $74,142
Flagged rows that lose money: 41.5%   Baseline: 18.7%   Enrichment: 2.22x
Reproducible: identical flags on repeat runs (random_state=7)
```

The finding: our own justification for the forest was wrong

The standard argument for an Isolation Forest is that it catches rows whose *combination* of values is unusual while no single column is extreme. **We measured this. On this dataset it is false.**

Contamination	Flagged	Caught by forest but <i>not</i> by IQR
1%	100	0
2%	200	0
5%	500	1
10%	1,000	22

At every operationally sensible threshold, the forest finds **nothing** the univariate rules miss.

Its real value is selectivity. The union of the per-column IQR rules flags **2,851 rows** — **28.5% of the dataset**. “A third of your orders are unusual” is not an alert list anyone can act on; it is noise with a threshold attached. The forest returns a **bounded, ranked** set with a continuous score, so the worst row comes first and a human can start at the top. **IQR cannot rank at all.**

We report this rather than quietly selecting the algorithm that sounds most advanced. The test that actually matters — are flagged rows more likely to be genuinely bad? — the forest passes at **2.22× enrichment over baseline**.

The LLM never decides what is anomalous. It narrates rows the detectors have already flagged. A model that could invent anomalies could invent a crisis that does not exist.

7.2 D4 — ReAct multi-turn reasoning agent

A tool-calling loop: the model decides which tool to call, observes the result, and decides again. The full reasoning chain is surfaced in a collapsible debug panel.

Six tools against a required three:

```
query_data · detect_anomalies · get_holidays · compare_dates · create_chart ·
                                final_answer
```

Why the agent’s tools need no sandbox — and Task B’s do

This is the design distinction worth the most in this project.

	Task B pipeline	Task D4 agent
Model produces	Arbitrary pandas code	Typed tool arguments
Who builds the call	The model	The executor
Safety model	Containment (3-layer sandbox)	Construction (schema-bounded)
Can express	Anything Python can	Only what the tool schema allows

The agent’s `query_data` takes an enum’d column name and an enum’d aggregation. The model *cannot express* anything outside that schema, so this surface is **safe by construction and needs no sandbox at all**. Both approaches exist in this project on purpose: one is right when you need expressive power, the other when you need guarantees.

The one tool that leaves the machine

The dataset is four years of US order lines and nothing else. It cannot say whether 24 November 2016 was a Thursday like any other or Thanksgiving — and “*do sales move around public holidays?*” is a fair question to ask of retail data that the data alone cannot answer. `get_holidays` fetches that missing calendar from Nager.Date, a free keyless public-holiday API; `compare_dates` then joins the returned dates back to the orders.

Network reach is where agents usually acquire their worst failure mode: a tool returns attacker-controlled prose, the prose enters the model’s context, and the model follows it — indirect prompt injection. Rather than abandon the safety-by-construction principle, we applied it:

Guard	Effect
Fixed host	The URL is built from constants. The model supplies a year and nothing else — it cannot name a host, path or scheme
Validated year	Must be an integer inside the dataset’s 2014–2017 span
Two fields kept	Only an ISO date and a name survive; every other field of the response is dropped
Name sanitised	Whitespace collapsed, truncated to 60 chars — so a name cannot forge extra lines inside the observation block
Bounded	8s timeout, hard cap on holidays returned
Cached	Memoised per year: fetched once per process
Fails soft	Network errors become an <i>observation</i> , not an exception — a dead API cannot end a run

This API was chosen over a web search precisely because its response has no free-text field worth attacking — dates and short names, nothing else. Search would have looked more impressive and handed the model a paragraph of someone else’s text. Verified against a hostile payload (a holiday “name” carrying newlines and IGNORE ALL PRIOR RULES): the forged newlines are collapsed, the extra fields dropped, and one truncated line reaches the model.

The residual risk, stated rather than hidden: up to 60 characters from a fixed, known host still reach the model. That is not zero. It is acceptable here on quantity and source, and it would not be for a search tool.

Live results (real Gemma 4 E4B)

```
Q: "Which region has the highest total sales? Chart it."
ACT query_data({"aggregation":"sum","group_by":"Region","metric":"Sales"})
OBS 4 rows in 3ms: West 725457.82 | East 678781.24 | Central 501239.89 | South 391721.91
ACT create_chart({"chart_type":"bar"})
ANS "The West region has the highest total sales with $725,457.82."
-> 2 tool calls, 2.3 s. Correct.

Q: "Find the most unusual order lines and tell me what went wrong."
ACT detect_anomalies({"top_n": 5})
OBS Isolation Forest flagged 200 of 9,994 (total loss on flagged: -$74,142)
THINK "CA-2016-117121 (Binders): Profit was $4,946.37. This line does not
      appear to show a loss based on the provided data."
ANS "...CA-2016-108196 (Machines) shows a profit of -$6,599.98..."
-> 1 tool call, 10.2 s.

Q: "Do sales rise around US public holidays in 2016?"
ACT get_holidays({"year": 2016})
OBS 13 US public holidays in 2016 (from date.nager.at): 2016-01-01 New Year's Day ...
ACT compare_dates({"dates": [13 dates], "metric": "Sales", "window_days": 3})
OBS those dates: $2,133.15/day across 73 days | other days: $1,839.76/day across 1,164
    difference: +15.9% per day
ANS "Yes, sales appear to rise around US public holidays in 2016..."
-> 2 tool calls, 7.4 s. Correct.
```

Three things in that transcript are worth pointing at. In the second run the agent **correctly observed that the top-ranked anomaly is not a loss at all** — it is an unusually large *profit* — and said so before moving on to the ones that are. That is reasoning about an observation, not pattern-matching. In the third, the second call’s arguments *are the first call’s output*: a genuinely dependent chain, which is precisely what the Task B pipeline cannot express and the reason the agent exists. And asked the same question about **2026**, the agent answers “the available data only covers up to 2017” and offers the years it does have — it declines rather than fabricates.

Stated limitation. Gemma 4 E4B plans two or three steps competently and then begins to loop, re-calling the same tool with the same arguments. `MAX_STEPS=6` and duplicate-call detection are guards a larger model would not need. We report this rather than hiding it behind a curated demo.

8 Evaluation

What works well

- **Text-to-code accuracy: 10/10** on the benchmark, 2.0s mean, zero retries.
- **The sandbox holds.** 8/8 escape attempts blocked, including every bypass that defeats a naive substring filter.
- **Sub-5 ms queries**, 100× under budget.
- **Zero cost per query**, no data leaves the machine.
- **Colour is accessible**, validated rather than assumed.
- **The agent genuinely composes** multi-step plans, reaches an external API, and reasons about observations.

What does not

- **There is no automated test suite.** The `check_task_*.py` scripts are acceptance smoke tests that need a live model; they are not unit tests and no CI runs them. This is the project’s largest engineering gap, and it is not theoretical — see below.
- **Two bugs reached the deployed app** (§6.5): the kaleido/Chrome crash, and the base64 array encoding that broke every question after the first. Both were in pure functions, both were trivially testable, and both reached production because nothing ran before the push.
- **The agent loops** beyond 2–3 steps. Guarded, not solved.
- **The agent is narrow.** Its typed tools cannot express what `query_data`’s enums do not contain, so for most questions the Task B pipeline is the better tool. The agent earns its place on external-data questions and dependent chains, which is a real but narrow win.
- **Cloud reports omit the map.** Bar, line and scatter fall back to matplotlib; a choropleth cannot be redrawn without Plotly’s geometry.
- **The stacked bar has no drill-down; dark mode is built and validated but not exposed; responsiveness at 1280px is untested.**
- **The Isolation Forest adds no novelty over IQR** on this dataset (§7.1). Its value is ranking and selectivity, which is a weaker claim than we expected to make.
- **Two benchmark questions were reworded** after observing failure (§5.4). We believe this was justified; we disclose it so the reader can judge.
- **A performance bug nearly wrecked the demo.** `st.download_button` evaluates its `data` argument *eagerly* — each export rendered a chart through kaleido at ~1.5s. With five answers in history, **every filter change triggered 30.7s of work**. Found by profiling the rerun path; fixed by caching against the filter scope. We found this by *measuring* rather than by assuming the code was fine because it looked fine.

Honest assessment of the model

Gemma 4 E4B is **good enough**, but only because the system is built around its weaknesses. Every one of the three defects we fixed (§5.4) is a defect a frontier model would not have exhibited. The correct conclusion is not “4B models are sufficient” — it is that **small-model reliability is an engineering problem, and the engineering is tractable**.

9 Future Work

1. **An automated regression test suite.** The two bugs that reached the deployed app were both in pure functions: chart-fallback rendering and array decoding. A pytest suite over `export`, `browserless` (asserting through a pickle round trip, which is what Streamlit’s cache actually does), `external`’s validation and sanitising, `query.aggregate` and `autochart.select_chart_type`

needs no model, runs in about a second, and would have caught both before the push. This is the highest-value item on the list and the one we would do first.

2. **Semantic caching of generated code.** Questions repeat. Embedding the question and reusing a previously validated snippet for near-duplicates would cut mean latency from 2.0s to near-zero on the common path, and remove the largest remaining demo risk.
3. **Self-consistency for high-stakes queries.** Generate the code three times at temperature 0.3, execute all three, and return the answer only if they agree. Trades $\sim 3\times$ latency for a measurable reduction in silent wrong answers — the most dangerous failure mode, because a plausible wrong number is worse than an error.
4. **Process-level sandbox isolation.** The current sandbox is language-level, which is right for a trusted local model but insufficient for untrusted users. A subprocess with a seccomp profile and a memory cap would make the endpoint safe to expose.
5. **A larger model behind the agent.** The ReAct loop is bottlenecked by planning depth, not by tools. The same six tools with a 20B+ model would plan five or six steps without looping and would not need MAX_STEPS or duplicate detection.

10 Conclusion

We set out to test whether a 4-billion-parameter model on consumer hardware can power a genuine natural-language analytics platform. It can — reaching **10/10** on a benchmark scored against hand-written ground truth, at **2.0 s per question**, at **zero cost per query**, with **no data leaving the machine**.

But the headline number is the least interesting result. The benchmark began at **5/10**, and every point of improvement came from reading the model’s actual output and diagnosing a specific defect: a JSON-escaping interaction that silently corrupted generated code; a missing few-shot pattern that made the model answer a subtly different question; and a bug in our own sandbox where two defence layers had drifted out of agreement. **None of it came from a bigger model.**

The same pattern recurred in the analysis itself. The Isolation Forest was chosen for a reason that turned out, on measurement, to be false — and the honest finding underneath it (selectivity, not novelty) is more useful than the one we expected. It recurred again in deployment, where the most expensive dependency in the build turned out to be a browser we only needed for four download buttons.

The lesson we take from this project is that **working with a small model is not a compromised version of working with a large one. It is a different discipline** — one where the model’s failures are legible, reproducible, and fixable, and where the engineering is the substance rather than the scaffolding.

11 AI Usage Disclosure

Required by §8 of the brief.

This project was developed with **Claude (Anthropic)** used as an AI coding assistant, in the manner permitted by the brief’s §8 (“permitted for code completion and debugging”). We disclose its use broadly rather than narrowly: it was a substantial contributor to the code and to the drafting of this report, and describing it as mere autocomplete would be untrue.

How it was used.

- Scaffolding the module structure and boilerplate.
- Drafting implementations of the data layer, prompt templates, the AST sandbox, the chart suite, and the export paths.

- Diagnosing the benchmark defects described in §5.4. In each case the *failure* was surfaced by our own acceptance-check scripts, and the diagnosis was reached by reading the code the model actually emitted.
- Designing and implementing the deployment path in §3.1 — the authenticating proxy, the tunnel, and the start/stop scripts.
- Diagnosing and fixing the two deployed-app crashes in §6.5, and writing the browserless chart fallback.
- Implementing the agent’s external holiday tool and its injection guards (§7.2).
- Drafting this report.

What the team did.

- Selected the dataset (D1) and the model, and justified both (§2.2, §5.1).
- Ran every acceptance check and every benchmark run on the LM Studio machine; all figures quoted in this report are from those runs.
- Made and own the design decisions the report defends: the refusal to build a dual-axis chart (§6.3), rule-based rather than LLM-based chart selection, the two different safety models for Task B and Task D4 (§7.2), the choice of a structured API over a web search for the agent’s network tool, and the interpretation of the Isolation Forest result (§7.1).
- Interpreted the analytical findings.

On the brief’s §8 requirement that every line be explainable. The brief states that every team member must be able to explain any line of submitted code, and that the Q&A will probe this directly. We regard that as a standard to have met before the demo, not a box to tick. The five points every member should be able to answer without notes are: how the AST allowlist defeats `__import__("os")` where a substring check does not; what the three phases of the NL pipeline are and what happens when Phase 2 fails; why the Isolation Forest is retained even though it catches nothing the IQR rules miss; why there is no dual-axis chart; and why the agent’s tools need no sandbox while Task B’s do.

References

- [1] Superstore Dataset. Kaggle. <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>
- [2] Liu, F.T., Ting, K.M., Zhou, Z-H. (2008). *Isolation Forest*. IEEE ICDM.
- [3] Yao, S. et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR.
- [4] Gemma Team, Google DeepMind. *Gemma model card*.
- [5] LM Studio documentation — OpenAI-compatible endpoints. <https://lmstudio.ai/docs>
- [6] Streamlit documentation — Community Cloud deployment and secrets management. <https://docs.streamlit.io>
- [7] Cloudflare Tunnel documentation. <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>
- [8] Plotly and Kaleido documentation — static image export. <https://plotly.com/python/static-image-export/>
- [9] Nager.Date — public holiday API. <https://date.nager.at/Api>
- [10] Matplotlib documentation — the Agg backend. <https://matplotlib.org/stable/users/explain/figure/backends.html>